Hybrid Branch Prediction with MC Dropout: Comparing {Gshare, Bimodal, 2-Bit} - BNN

Sebastian Candelaria, Eshgin Hasanov, Abram Tadros, Yousef Awad
Department of Computer Science
University of Central Florida
Orlando, Florida, USA
{sebastian.candelaria, eshgin.hasanov, abram.tadros, yousef.awad}@ucf.edu

***Abstract*—Modern processors depend on highly accurate branch predictors to sustain high instruction throughput while minimizing energy consumption and pipeline stalls caused by mispredictions. Traditional predictors like the 2-bit saturating counter are power-efficient but limited in handling complex, correlated branch patterns, whereas advanced approaches such as deep reinforcement learning models (e.g., neural networks) achieve higher accuracy at the cost of substantial computational overhead and power draw. Recent work has demonstrated that hybrid predictors often provide the best trade-off; for instance, a study published in IEEE introduced a system combining a Bayesian Neural Network-based Deep Q-Network (BNN-DQN) with a conventional 2-bit counter, dynamically routing simple branches to the low-power counter and invoking the neural model only when prediction confidence is low [1]. This research expands that hybrid framework by evaluating standard 2-bit, Bimodal, and Gshare baselines alongside a Bayesian Neural Network utilizing Monte Carlo Dropout (BNN-MCD) to approximate predictive uncertainty. By testing both simple and confidence-gated arbitration policies, this study investigates whether stronger baseline predictors can reduce reliance on the neural fallback and mitigate destructive interference. By improving the baseline predictor's accuracy on straightforward cases, the proposed design aims to restrict unnecessary activations of the neural component, which can actively degrade overall accuracy when the model is poorly converged. Ultimately, this work seeks to determine the optimal balance between hardware baseline strength and neural intervention to enable highly accurate, robust branch prediction in next-generation high-performance processors.**

I. Introduction and Background

Accurate branch prediction remains essential for maintaining high instruction throughput in deeply pipelined processors. Simple predictors such as 2-bit saturating counters are inexpensive and effective on regular branch behavior, but they often struggle when control flow depends on longer or less obvious patterns. More advanced predictors such as gshare improve prediction quality by combining program counter information with global history, although they still face accuracy and aliasing limits on difficult traces. Neural predictors can capture richer correlations, but invoking them on every branch increases runtime and implementation cost.

This project studies a hybrid design that uses a low-cost baseline predictor first and calls a Bayesian-style neural network only when the baseline appears uncertain. The neural component uses Monte Carlo dropout to approximate predictive uncertainty, which gives the design a practical way to apply neural prediction selectively instead of universally [2]. We compare three baselines under the same hybrid policy: 2-bit + BNN, bimodal + BNN, and gshare + BNN.

The main research question is whether replacing a simple 2-bit baseline with stronger table-based predictors such as bimodal or gshare improves the quality of a hybrid predictor when all configurations share the same neural fallback, feature encoding, and evaluation methodology. The null hypothesis is that a stronger baseline should improve the hybrid predictor by correctly handling straightforward branches and reducing the number of cases where the neural fallback can interfere. The challenge is that the BNN is trained online during trace execution, so under limited warmup and simulation windows it may not converge enough to reliably improve weak-counter cases.

Our evaluation shows that this interaction is trace-dependent. In several of our ChampSim runs, non-neural baseline predictors match or outperform their BNN hybrid counterparts, indicating that neural fallback is not automatically beneficial when online training is limited. At the same time, the confidence- gated policy can slightly improve selected predictor/trace combinations by preventing uncertain BNN predictions from overriding the baseline. Our novelty is therefore the controlled comparison of 2-bit, bimodal, and gshare baselines inside the same uncertainty-gated neural framework, which makes the tradeoff between baseline strength, neural assistance, and online convergence easier to evaluate directly.

II. Design and Implementation

A. Hybrid Predictor Structure

This implementation is inspired by prior work on hybrid and neural branch prediction [3], [4], but it is intentionally simpler than a full Bayesian DQN-style design. In particular, the current repository trains the neural component online with supervised branch labels, rather than with rewards, Bellman updates, or a separate target network.

All proposed predictors use the same two-stage organization. A conventional counter-based predictor first produces a branch direction and an implicit confidence value from its saturating-counter state. Strong counter states are used directly, since those branches are likely to be handled well by a low-cost predictor. Weak counter states expose the hard cases: branches whose recent behavior has not settled into a clear

taken or not-taken direction. For those branches, the hybrid design can consult a Bayesian-style neural fallback.

The two evaluated designs differ only in the baseline predictor. The 2-bit + BNN uses a single global 2-bit saturating counter similar to the BNN-DQN implementation [1]. Each counter predicts not taken in states 0 and 1 and taken in states 2 and 3; states 1 and 2 are treated as weak. The bimodal + BNN configuration uses a PC-indexed table of 2-bit saturating counters, which corresponds to the standard bimodal form of a 2-bit predictor. The gshare + BNN configuration uses the same 2-bit counter states, but indexes its table by XORing low-order PC bits with a global branch history register. This gshare index allows branches with similar PC bits to be distinguished by the recent control-flow context in which they occur.

B. MC-Dropout Neural Fallback

The neural component is an online classifier that predicts the probability that the current branch will be taken. Its feature vector contains signed encodings of PC bits and global-history bits. In the ChampSim implementation, we also include baseline context: the raw counter value, the baseline direction, and a weak-state flag. These extra features allow the neural fallback to learn not only branch behavior, but also when the baseline predictor appears unreliable.

To approximate Bayesian uncertainty, the network uses Monte Carlo dropout [2]. Dropout remains active during inference, so the same feature vector is evaluated for S stochastic forward passes. The mean probability is

$$\bar{p} = \frac{1}{S}\sum_{i=1}^{S} p_i, \tag{1}$$

and the neural predictor returns taken when $\bar{p} \geq 0.5$. The variation among the S samples is used as an uncertainty estimate. This is a practical Bayesian approximation rather than a full BNN-DQN: the model is trained with supervised taken/not-taken labels and uses dropout-derived uncertainty for arbitration.

C. Arbitration Policies

We implemented two arbitration policies for each baseline, giving four ChampSim predictor variants. The simple policy follows the original hybrid intuition: whenever the baseline counter is weak, the BNN output replaces the baseline output. The confidence-gated policy adds a second condition:

$$\text{useBNN} = \text{weak}(\text{base}) \wedge \text{confident}(\text{BNN}). \tag{2}$$

Thus, a weak baseline does not automatically force a neural override. If the BNN is also uncertain, the predictor falls back to the baseline direction. This policy is motivated by the fact that a neural model trained online may be poorly calibrated early in a trace or during phase changes. Avoiding low-confidence neural overrides can reduce cases where the fallback predictor hurts an otherwise correct baseline prediction.

D. Online Training and ChampSim Integration

Training occurs online during trace execution. After the true branch outcome is known, the baseline counter is updated toward the observed direction and the saved neural feature vector is inserted into a replay buffer. After an initial warmup period, the neural predictor periodically samples minibatches from this buffer and updates its weights with binary classification loss. No offline pretraining, reward signal, target network, or Bellman update is used.

The final implementation is packaged as ChampSim branch-predictor modules. The simulator calls `predict_branch` to obtain a direction prediction and `last_branch_result` when the branch resolves. Because ChampSim models out-of-order execution, the predictor stores prediction-time state for each in-flight branch, including the feature vector, baseline decision, neural prediction, and final hybrid decision. For gshare, the implementation also saves the exact counter index computed during prediction, so resolution updates the same table entry even if the global history has changed by the time the branch commits.

The predictor maintains separate speculative and committed global histories. Speculative history is updated with predicted outcomes so later predictions can use the predicted control-flow path. Committed history is updated only when branches resolve. On a misprediction, speculative history is restored from the committed state. This separation prevents the baseline and neural components from being trained on feature states that did not match the original prediction.

E. Reported Metrics

The ChampSim simulator experiments report branch prediction accuracy, MPKI, IPC, and per-branch-type MPKI for the region of interest. Together, these outputs allow us to compare prediction quality between 2-bit + BNN, gshare + BNN, and bimodal + BNN.

III. Evaluation and Results

A. Experimental Setup

To evaluate the accuracy of the proposed hybrid branch predictor configurations, we utilized the ChampSim simulator, a trace-based microarchitectural modeling environment. The evaluation workloads consisted of integer benchmarks from the SPEC CPU 2006 suite, specifically focusing on the branch-heavy traces 403.gcc and 600.perlbench. Each simulation was initialized with a warmup period of 2 million instructions to prime the architectural state, followed by a detailed simulation phase of up to 5 million instructions.

We evaluated three baseline architectures: a standard 2-bit saturating counter, a Bimodal predictor, and a Gshare predictor. For each baseline, we compared three operational policies:

- **Non-BNN:** The baseline hardware predictor using the built-in models from ChampSim.
- **BNN-Simple:** A hybrid model where the Bayesian Neural Network (BNN) overrides any branch the baseline considers "weak" (counter states 1 or 2).

- **BNN-Complex:** A hybrid model leveraging Monte Carlo Dropout (BNN-MCD) that overrides weak branches only when the BNN uncertainty metric falls below a strict confidence threshold of 0.3.

The BNN was trained online during trace processing. Based on empirical stabilization tests, the BNN hyperparameters were tuned to a learning rate of 1×10^{-3} and a dropout rate of 0.3 to prevent catastrophic forgetting and maintain pattern recognition on complex workloads.

B. Results

The overall accuracy results across the tested traces are summarized in Table I.

TABLE I
ACCURACY COMPARISON OF HYBRID BNN AND BASELINE PREDICTORS.

Predictor	Trace	BNN-Simple	BNN-Complex	Non-BNN
Gshare	600.perlbench	94.37%	94.50%	97.11%
Gshare	403.gcc	98.54%	98.44%	99.54%
Bimodal	600.perlbench	97.44%	97.40%	97.04%
Bimodal	403.gcc	99.40%	99.39%	99.40%
2-Bit	600.perlbench	71.75%	72.12%	96.99%
2-Bit	403.gcc	98.09%	98.09%	99.38%

Our analysis indicates that while BNN integration provides high peak accuracy, particularly on the 403.gcc trace where the BNN-Bimodal configuration reached 99.40%, the neural component requires careful baseline alignment. The most significant performance gap was observed in the 2-bit counter on the perlbench trace, where the baseline Non-BNN accuracy of 96.99% dropped to 72.12% under the BNN-Complex policy. The results suggest that the neural model's performance is highly sensitive to its configuration. Achieving a net gain in accuracy requires a more rigorous optimization of the model's parameters. Specifically, the learning rate and uncertainty thresholds must be precisely tuned to the workload to ensure that neural overrides consistently improve upon the baseline's hardware predictions.

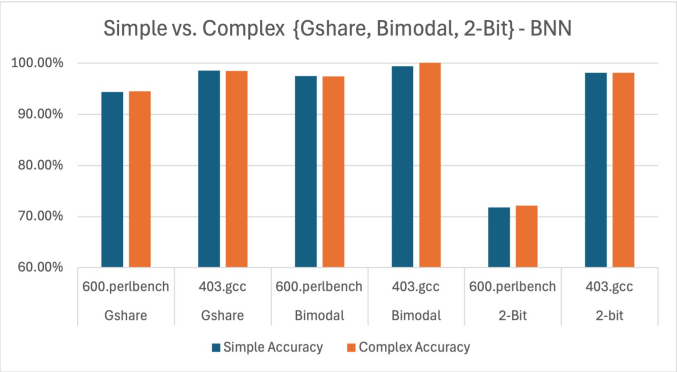

Fig. 1. Simple vs. Complex Gshare, Bimodal, 2-Bit - BNN

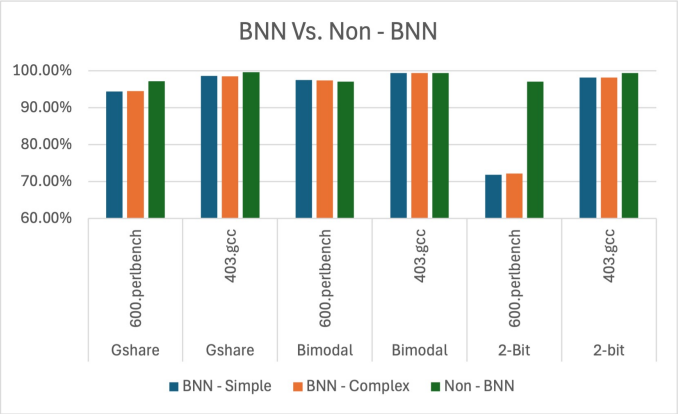

Fig. 2. BNN vs. Non-BNN

IV. CONCLUSION

This project evaluates three hybrid branch predictors that use the same Bayesian-style neural fallback but different conventional baselines. Replacing the 2-bit base predictor with G-Share and Bimodal coounterparts greatly increased prediction accuracy, indicating that using more complex predictors can decrease reliance on BNN effectiveness. However, the non-hybrid predictors performed better than hybrid predictors on most traces. This would suggest that further work must be done on configuring parameters of the BNN, such as learning rate and uncertainty thresholds, to increase BNN accuracy and overall performance. Additionally, testing on longer traces with an actual BNN-DQN implementation could allow the BNN to learn more quickly and provide accurate results earlier in trace runs.

Overall, in terms of accuracy, hybrid BNN configurations using stronger base predictors surpass the standard 2-bit implementation but can struggle against regular predictors without fine-tuned parameters. Future works should explore correlations between different BNN parameters and increased prediction accuracy to determine the best BNN models for branch prediction.

REFERENCES

[1] A. M. Nair, A. Pananjikkal Sujeendran, and R. K. Megalingam, "Branch prediction using reinforcement learning," in *2025 International Conference on Robotics and Mechatronics (ICRM)*, 2025, pp. 1–5.

[2] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning," in *Proceedings of the 33rd International Conference on Machine Learning*, 2016, pp. 1050–1059.

[3] S. McFarling, "Combining branch predictors," Digital Equipment Corporation, Western Research Laboratory, Tech. Rep. TN-36, 1993.

[4] D. A. Jimenez and C. Lin, "Dynamic branch prediction with perceptrons," in *Proceedings of the 7th International Symposium on High-Performance Computer Architecture*, 2001, pp. 197–206.